Specifications	
Addressing capacity (bits)	**16**
RAM (kibibits)	**512**
RAM (kibibytes)	**64**
Clock frequency	**2 MHz**
Stack pointer (bits)	**16**
Stack pointer at reset	**0xFE00**
Interrupt Vectors Table (bytes)	**512**
Instruction min size (bytes)	**1**
Instruction max size (bytes)	**4**
Number of instructions	**54**
Number of addressing modes	**9**

Flag Register (8b)	
Carry (C)	0
Equal (E)	1
Interrupt (I)*	2
Negative (N)	3
Superior (S)	4
Zero (Z)	5
Inferior (F)	6
Halt (H)	7

** Flag up when interrupts are handled*

Instruction format (4 bytes)				
OpCode (6b)	**A.M. (4b)**	**Regs (6b)**	**V1 (8b)**	**V2 (8b)**

The CPU fetches the bytes it needs, depending on the opcode, then addressing mode, allowing 1, 2, 3 instructions as well.

RAM format	Address
Program memory & data (code, global variables, heap ↓)	0x0000

Free memory	---

Stack ↑ (grows toward 0x0000) first push at 0xFDFF	---

	0xFDFF
Interrupt Vectors Table (512 bytes)	0xFE00

	0xFFFF

Registers		
PC (16b)	*Prog. Counter*	
STK (16b)	*Stack Pointer*	
A	0x0	
B	0x1	
C	0x2	
D	0x3	
I	0x4	
J	0x5	
X	0x6	
Y	0x7	

Notation: Rx = register pair R1:R2 (R1 = MSB) · $(…) = memory at address · V1/V2 = 8-bit values, Vx = 16-bit value · STK grows downward, points to the last byte pushed

Addressing modes (4b)	
None	0
Reg	1
Reg / Imm8	2
Reg / Ram	3
RamReg / ImmReg / Offset	4
Stack / Offset	5
Reg16	6
Imm16	7
Imm8	8

Instructions	A.M.	DEC	HEX	BIN	Affected Flags	Instruction Length (bytes)
NOP	0	0	**0x00**	000000		1
ADC — R1 + R2 + C → R1	1	1	**0x01**	000001	C, Z, N	2
ADC — R1 + V1 + C → R1	2					3
ADC — R1 + $(Vx) + C → R1	3					4
ADC — R3 + $(Rx + V2) + C → R3	4					4
ADC — R1 + $(STK + V1) + C → R1	5					3
ADD — R1 + R2 → R1	1	2	**0x02**	000010	C, Z, N	2
ADD — R1 + V1 → R1	2					3
ADD — R1 + $(Vx) → R1	3					4
ADD — R3 + $(Rx + V2) → R3	4					4
ADD — R1 + $(STK + V1) → R1	5					3
AND — R1 & R2 → R1	1	3	**0x03**	000011	Z, N	2
AND — R1 & V1 → R1	2					3
AND — R1 & $(Vx) → R1	3					4
AND — R3 & $(Rx + V2) → R3	4					4
AND — R1 & $(STK + V1) → R1	5					3
CAL — push PC, Rx → PC	6	4	**0x04**	000100		2
CAL — push PC, Vx → PC	7					4
CLC	0	5	**0x05**	000101	C	1
CLE	0	6	**0x06**	000110	E	1
CLI	0	7	**0x07**	000111	I	1
CLN	0	8	**0x08**	001000	N	1
CLS	0	9	**0x09**	001001	S	1
CLZ	0	10	**0x0A**	001010	Z	1
CLF	0	11	**0x0B**	001011	F	1
CMP — R1 ? R2	1	12	**0x0C**	001100	S, E, F	2
CMP — R1 ? V1	2					3
CMP — R3 ? $(Rx + V2)	4					4
CMP — R1 ? $(STK + V1)	5					3
DEC — R1-- → R1	1	13	**0x0D**	001101	C, Z, N	2
DEC — $(Rx + V2)-- → $(Rx + V2)	4					4
DEC — $(Vx)-- → $(Vx)	7					4
HLT	0	14	**0x0E**	001110	H	1
IN — Port(R2) → R1	1	15	**0x0F**	001111		2
OUT — R2 → Port(R1)	1	16	**0x10**	010000		2
INC — R1++ → R1	1					2

Instructions	A.M.	DEC	HEX	BIN	Affected Flags	Instruction Length (bytes)
INC — $(Rx + V2)++ → $(Rx + V2)	4	17	**0x11**	010001	C, Z, N	4
INC — $(Vx)++ → $(Vx)	7					4

Instructions	A.M.	DEC	HEX	BIN	Affected Flags	Instruction Length (bytes)
INT – push I, push PC, $(0xFE00 + 2×V1) → PC	8	18	**0x12**	010010	I	3
IRT – pop PC, pop I	0	19	**0x13**	010011	I	1
JMC – (Rx) → PC	6	20	**0x14**	010100		2
JMC – (Vx) → PC	7					4
JME – (Rx) → PC	6	21	**0x15**	010101		2
JME – (Vx) → PC	7					4
JMN – (Rx) → PC	6	22	**0x16**	010110		2
JMN – (Vx) → PC	7					4
JMP – (Rx) → PC	6	23	**0x17**	010111		2
JMP – (Vx) → PC	7					4
JMS – (Rx) → PC	6	24	**0x18**	011000		2
JMS – (Vx) → PC	7					4
JMZ – (Rx) → PC	6	25	**0x19**	011001		2
JMZ – (Vx) → PC	7					4
JMF – (Rx) → PC	6	26	**0x1A**	011010		2
JMF – (Vx) → PC	7					4
STR – R1 → $(Vx)	3	27	**0x1B**	011011		4
STR – R3 → $(Rx + V2)	4					4
STR – R1 → $(STK + V1)	5					3
LOD – $(Vx) → R1	3	28	**0x1C**	011100		4
LOD – $(Rx + V2) → R3	4					4
LOD – $(STK + V1) → R1	5					3
MOV – R2 → R1	1	29	**0x1D**	011101		2
MOV – V1 → R1	2					3
NOT – ~R1 → R1	1	30	**0x1E**	011110	Z, N	2
NOT – ~$(Vx) → $(Vx)	7					4
OR – R1 or R2 → R1	1	31	**0x1F**	011111	Z, N	2
OR – R1 or V1 → R1	2					3
OR – R1 or $(Vx) → R1	3					4
OR – R3 or $(Rx + V2) → R3	4					4
OR – R1 or $(STK + V1) → R1	5					3
POP – $(STK) → R1, STK + 1 → STK	1	32	**0x20**	100000		2
PSH – STK – 1 → STK, R1 → $(STK)	1	33	**0x21**	100001		2
RET – pop PC	0	34	**0x22**	100010		1
SHL – R1	1	35	**0x23**	100011	C, Z, N	2
ASR – R1	1	36	**0x24**	100100	C, Z, N	2

Instructions	A.M.	DEC	HEX	BIN	Affected Flags	Instruction Length (bytes)
SHR – R1	1	37	**0x25**	100101	Z, N	2
STC	0	38	**0x26**	100110	C	1

Instructions	A.M.	DEC	HEX	BIN	Affected Flags	Instruction Length (bytes)
STE	0	39	**0x27**	100111	E	1
STI (after next instruction is done)	0	40	**0x28**	101000	I	1
STN	0	41	**0x29**	101001	N	1
STS	0	42	**0x2A**	101010	S	1
STZ	0	43	**0x2B**	101011	Z	1
STF	0	44	**0x2C**	101100	F	1
PSF — STK − 1 → STK, FR → $(STK)	0	45	**0x2D**	101101		1
POF — $(STK) → FR, STK + 1 → STK	0	46	**0x2E**	101110	All	1
SUB — R1 - R2 → R1	1	47	**0x2F**	101111	C, Z, N	2
SUB — R1 - V1 → R1	2					3
SUB — R1 - $(Vx) → R1	3					4
SUB — R3 - $(Rx + V2) → R3	4					4
SUB — R1 - $(STK + V1) → R1	5					3
SBB — R1 - R2 - C → R1	1	48	**0x30**	110000	C, Z, N	2
SBB — R1 - V1 - C → R1	2					3
SBB — R1 - $(Vx) - C → R1	3					4
SBB — R3 - $(Rx + V2) - C → R3	4					4
SBB — R1 - $(STK + V1) - C → R1	5					3
XOR — R1 ⊕ R2 → R1	1	49	**0x31**	110001	Z, N	2
XOR — R1 ⊕ V1 → R1	2					3
XOR — R1 ⊕ $(Vx) → R1	3					4
XOR — R3 ⊕ $(Rx + V2) → R3	4					4
XOR — R1 ⊕ $(STK + V1) → R1	5					3
ASP — STK + Vx → STK	7	50	**0x32**	110010		4
ASP — STK + V1 → STK	8					3
SSP — STK − Vx → STK	7	51	**0x33**	110011		4
SSP — STK − V1 → STK	8					3
RSP — STK → Rx	6	52	**0x34**	110100		2
WSP — Rx → STK	6	53	**0x35**	110101		2
WSP — Vx → STK	7					4